

CS 699: Software Lab

Om P. Damani (CSE)

What is this Course About

- developing software?
- programming?
- performance?
- architecture?
- tools?
- frameworks?
- philosophy?

What is this Course About

- ~~developing software?~~
- programming?
- ~~performance?~~
- ~~architecture?~~
- tools?
- frameworks?
- philosophy – end to end understanding

Course Objective

Training in the **Art** and **Science** of Software Development

Science is what we understand well enough to explain to a computer. Art is everything else we do.

- Donald Knuth

Logistics

SL 1-3

Access to lab outside lab hours???

Communication

- Moodle/Teams: (search for CS 699)
- Please enroll
- Matter of common interest on common channels, private interest on chat
- All emails should have CS 699 as first part of subject
- Email header should be as specific as possible – say ‘CS 699 Assignment 3

Office Hours

- Office Hours: Fix hour: 5-5.30, Wed (right after class, in classroom)
- By phone: whatsapp me on mobile 24x7
- Any time by appointment

Feedback: Two way feedback critical for success of the course

Feedback: Two way feedback critical for success of the course

- Feedback to the students
 - Lab Assignments/HW, Quizzes, Exam
- Feedback from the students
 - During the class – ask if something is not clear
 - Will help pace the lecture
 - Right after the class if a lecture was not clear at all
 - Will help decide if some concepts need to be rediscussed in the next lecture
 - While reviewing a lecture – make a list of questions that come to mind,
 - Use email for subjective feedback, always ask technical questions in person or on newsgroup
- Teaching evaluation forms

Evaluation

- Labs+Exam: 70%
- Semester-long Project: 30%
- Practice Exercises: 0
- Roughly one submission a week (3 hr workload)
- Semester-long Project
 - Select a topic and build on it (group of 3)
- Relative Grading
 - But all can get AA or FF
 - Focus on learning/exploration

Project

- Anything – sky's the limit
 - Very ambitious to modest
- What software project can you do if all software tools were available to you?
- What can you do this semester?
- Team formation – Aug 14
- Topic finalization – Aug 21
- Initial plan report – Aug 28

Class structure

- 3-hour lab with an included 30-90 min lecture
- Attendance compulsory – prior leave needed
- Attendance will be taken on the Safe app
- Mainly an exploratory course
 - Lecture part just to introduce a topic
 - You are expected to explore on your own
- Contact me/TAs in case of language difficulty

Academic Honesty

- Absolute must
- You will be allowed to discuss things but must credit the source if the idea/wording is not yours and not get marks for that part
- CSE Dept default policy:
 - malpractice in homework/lab: one grade penalty
 - malpractice in exam: fail
 - any second offence: a semester's suspension

Text-books/References

- World Wide Web (the World as your worksheet)
- AI???
 - As a Teaching Assistant
 - Assignment Project is not the end goal
 - Learning is !!!!
- and exams are also unfortunate reality !!

Non Credited Use of AI

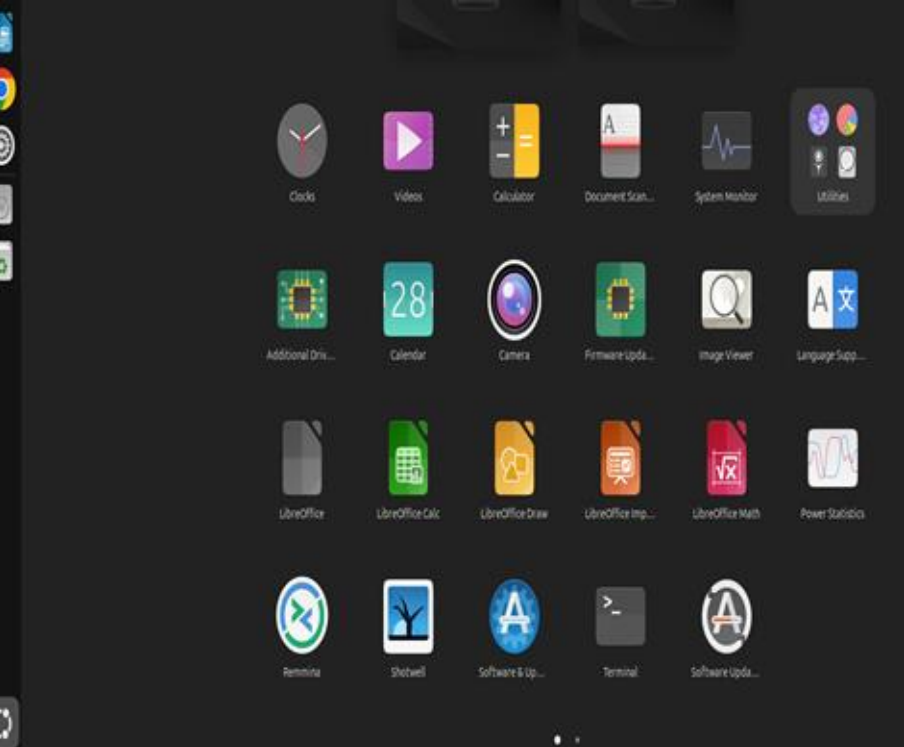
Last semester 1/3rd of the class got a zero in 25% weightage project

Many failed due to non-credited AI use or other academic dishonesty

Today's Class: Linux Shell Basics

- What is a Shell?
- Unix History/Linux Shell on Different Operating Systems
- Basic Operations
- Navigating the File System
- Listing Files and Directories
- Text Editors to Create Files
- Creating/Viewing/Moving/Modifying Files and Directories
- Miscellaneous cmd's
- Shell Scripting

Operating system: Graphical User Interface



Linux: GUI (Graphical User Interface)



Windows: GUI (Graphical User Interface)

Linux: Shell (Command-Line Interface)

```
sakina@cse: ~  
sakina@cse:~$ ip addr  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether a0:ad:9f:11:65:48 brd ff:ff:ff:ff:ff:ff  
    inet 10.129.142.188/24 brd 10.129.142.255 scope global dynamic noprefixroute  
        valid_lft 8470sec preferred_lft 8470sec  
    inet6 fe80::bb5b:39ad:85fc:954/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
sakina@cse:~$ ls -l  
total 48  
drwxr-xr-x 2 sakina sakina 4096 Nov 11  2025 Desktop  
drwxr-xr-x 2 sakina sakina 4096 Mar  5 10:14 Documents  
drwxr-xr-x 7 sakina sakina 4096 Apr 16 16:21 Downloads  
drwxrwxr-x 5 sakina sakina 4096 Feb 23 12:18 JTdevelopment  
drwxr-xr-x 2 sakina sakina 4096 Nov 11  2025 Music  
-rw-rw-r-- 1 sakina sakina  640 Feb 17 12:17 packages.microsoft.gpg
```

What is a Shell?

When you type: `pwd`

How does the computer know what to do?

Explanation: The **Shell** is a program that:

- Reads the command you type
- Interprets its meaning
- Runs the correct program
- Displays the output

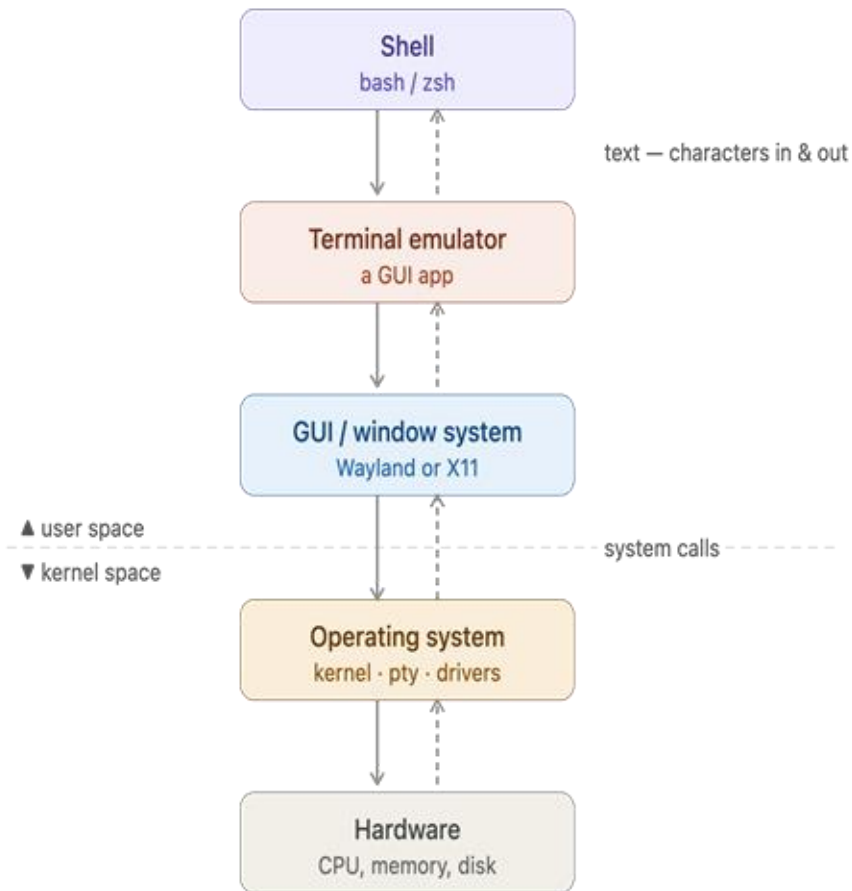
Think of it as the **command interpreter** between you and the operating system.

When you type `pwd` , The shell does not know what you want - it just executes the command.

Shell: Can be referred to / confused with

- **Shell**
- **Terminal**
- **Console**
- **Command Prompt (Prompt)**
- **Command Line Interface (CLI)**

Where does Shell reside?



Terminal Emulators

A terminal emulator is a GUI program that provides the text-based interface where your shell runs. There are many terminal emulators out there.

Some of the aspects that you may want to modify in your terminal include:

- Font choice

- Color Scheme

- Keyboard shortcuts

- Performance (some newer terminals like [Alacritty](#) or [Ghostty](#) offer GPU acceleration).

How do we use shell?

Different things we can do using shell **without clicking through folders**

- Open/create/delete file/folders
- Run programs/scripts
- Automate repetitive work: Check logs
- Process data/files

The shell lets developers interact with the operating system by typing commands.

The Shell also provides a scripting language and can perform these tasks in a few commands.

Unix History

Unix was originally developed for large **mainframe and multi-user computers**. Multiple users connected to the same computer using **remote terminals** (a keyboard and display).

- Terminals based on text since text is light on resources
- The terminal itself did **not run programs**; it only sent text commands to the computer and displayed the text output.
- Commands were intentionally kept **short and concise** (e.g., `ls`, `cp`, `mv`, `pwd`) to reduce typing and improve productivity.

Linux Shell on Different Operating Systems

- **Linux**
 - Shell is built into the operating system.
 - Common shells: **Bash**, **Zsh**, **Fish**.
- **Windows**
 - Use **Windows Subsystem for Linux (WSL)** to run a Linux environment.
 - Alternatively, install **Bash** for basic Bash commands.
- **macOS**
 - **Terminal** is built in.
 - Default shell is **Zsh** (Bash is also available).
- **Android**
 - Android is based on the Linux kernel.
 - A Linux shell can be accessed using apps such as **Termius** or through **ADB (Android Debug Bridge)**.

Basic Operation

- **Open the Shell:** Search for **Terminal** or Press **Ctrl + Alt + T**
- **Enter a Command:** Type a command at the shell prompt (**\$** or **%**) and press **Enter** to execute it.
- **Commands can accept options and arguments:**
 - **Command** – the program to execute.
 - **Options** – modify the behavior of the command.
 - **Arguments** – specify the file, folder, or input on which the command should operate.

Example: **command** **[options]** **[arguments]**

tar -zcvf lab1.tgz lab1/

Navigating the File System

The shell always starts in a **default working directory** (also called the **home directory**).

- **Home directory:**
 - **users:** `sakina@cse ~ %`
 - Example: `/home/sakina`
- The **pwd (Present Working Directory)** command displays the **current working directory**, i.e., the location where you are currently working.

Example:

```
sakina@cse ~ % pwd
```

Output: `/Users/sakina`

- Use the **cd** (**Change Directory**) command to move between folders.
- **Types of Paths**
 - **Relative Path:** Specifies the location relative to the current working directory.
 - **Absolute Path:** Specifies the complete path starting from the root directory.
- **Special Path Symbols**
 - **/** → Refers to the **root directory** (the top-most directory of the file system).
 - **~** → Refers to the **current user's home directory**.

Example:

cd Documents

Relative path

cd /home/sakina/Documents

Absolute path (Linux)

cd ~

Home directory shortcut

- **Press Tab** to automatically complete file/directory names - works with most shell commands
- Use the **man** command to view the manual (documentation) for any command.

Use **cd** to navigate directories and **man** to learn how any command works.

Listing Files and Directories

- Use the **ls (List)** command to display the contents of a directory.
- **Example: ls [OPTIONS] [FILES/DIRECTORIES]**

Common Options

- **ls -l** – Display files in **long listing format** (permissions, owner, size, date, etc.).
- **ls -a** – Display **all files**, including hidden files (files starting with **.**).

Special Directory Entries: Every directory contains at least two special entries:

- **.** (**dot**) – Refers to the **current directory**.
- **..** (**dot dot**) – Refers to the **parent directory**.

Other Useful Options

- **-X** – Sort files by **extension**.
- **-S** – Sort files by **size** (largest first).
- **-t** – Sort files by **modification time** (most recent first).
- **-R** – List files **recursively**, including all subdirectories.

Text Editors to Create Files

There are several text editors available in Linux to create and edit files.

Common Editors

- **Vi / Vim** – Powerful terminal-based editor for advanced users
- **Nano** – Simple and beginner-friendly terminal editor
- **gedit** – Graphical text editor (GUI-based)

Basic Editing Tasks

- Create a new file
- Write text into the file
- Edit or delete existing text
- Save the file
- Save and exit the editor
- Reopen the file for further editing

Example: `vim notes.txt`

Creating Files and Directories

mkdir – Make Directory

Syntax: **mkdir** [OPTION] DIRECTORY # Creates one or more directories.

- Eg: **mkdir cs699** # directory
- **mkdir cs699/data** #directory inside a directory
- **-p** – Creates parent directories if they do not already exist
 - a. **mkdir -p cs000/data**

Echo file/directory: The **echo** command displays text on the terminal

- Eg: **echo "Hello World"**
- Eg: **echo "Hello World" > data.txt** # echo has no knowledge of data.txt

> redirects the output to a file.

There are multiple ways to create a new file in Linux.

- Create an **empty file** using the `touch` command.
- **Examples:** `touch data.txt`

Searching for a File

Use the `find` command to search for files.

Examples: `find . -name "data.csv"`

Sample data for data.csv

name,dept,score,attendance,student_result

ghishu,CSE,88.0,95.0,pass

yamuna,EE,72.0,88.0,pass

sohum,CSE,91.0,,Pass

sakina,ME,65.0,90.0,pass

periyar,CSE,,92.0,fail

ramesh, EE,29,89,fail

Viewing File Contents

cat (Concatenate)

- The **cat** command displays the contents of a text file on the terminal.
- Originally named **concatenate** because it can combine and display multiple files together.
- Syntax: **cat [OPTION] FILE**

Common Options

- **-n** – Display file contents with **line numbers**.
- **-s** – Suppress (remove) repeated blank lines in the output.

Examples:

- **cat data.csv**
- **cat -n data.csv**
- **cat -s data.csv**

Viewing Large Files contents

For large files, use the **less** command instead of **cat** , to view few lines at a time

Examples: **less data.csv**

Inside **less**, we can navigate using:

- **↑ / ↓** : Move one line up or down
- **Page Up / Page Down** : Move one page at a time
- **Home** : Go to the beginning of the file
- **End** : Go to the end of the file
- **q** : Quit the viewer

less is better for reading large files because it allows interactive navigation without loading the entire file onto the screen.

Moving Files and Directories

mv (Move)

- The **mv** command is used to **move** or **rename** files and directories.
- `mv [OPTIONS] SOURCE DESTINATION`
- Both **SOURCE** and **DESTINATION** can be **files** or **directories**.
- `mv oldname.txt newname.txt` – using mv to **Rename** a file/directory

Common Options

- **-i** – Interactive mode; prompts before overwriting an existing file.
- **-n** – Never overwrite an existing file.
- **-f** – Force the move without asking for confirmation.

Examples: `mv data.csv backup/`

#move file in the same directory

- `mv data.txt ../backup/`
- `mv file1.txt file2.txt backup/`
- `mv source_directory destination_directory/`

#Move a file into a parent directory

#Move multiple files

#Move a directory into another directory

Copying and Removing Files

cp (Copy)

- The **cp** command is used to **copy files and directories**.
- Similar to **mv**, except it **creates a copy** instead of moving the original.
- **cp [OPTIONS] SOURCE DESTINATION**

Common Options

- **-R** – Copy directories **recursively**, including all files and subdirectories.
- **-i** – Prompt before overwriting an existing file.
- **-p** – Preserve the original file's permissions, ownership, and timestamps.

Examples:

- **cp data.csv backup.csv,**
- **cp -R project backup_project**

rm – Use with Caution

Important Note:

- The **rm** command **permanently deletes** files and directories.
- Unlike Windows or macOS, **rm does not move files to the Recycle Bin/Trash.**
- Once deleted, files are **generally difficult or impossible to recover.**

The **rm** command permanently **deletes files.**

- **Use with caution**—deleted files cannot be easily recovered.
- `rm [OPTIONS] FILE`

Common Options

- **-i** – Prompt for confirmation before deleting.
- **-r** – Recursively delete directories and their contents.
- **-f** – Force deletion without confirmation.

Examples: `rm data.csv`

- `rm -i data.txt/`
- `rm -r cs699/`

`rmdir` (Remove Directory)

- Removes an **empty directory** only.
- If the directory contains files, use `rm -r (recursive)` instead.
- `rm -d` removes an **empty directory**, similar to `rmdir`.
- Example: `rm -d empty_folder`

Example: `rmdir empty_folder`

Be Careful When Using

`rm`

- Double-check the file or directory name before deleting.
- Be especially careful when using **wildcards** (`*`, `?`), as they may delete multiple files at once.
- Use the `-i (interactive)` option to confirm each deletion.

Misc

1. Grep , awk, sed, find, which.
1. A complete function using the shell cmds.

Navigation Commands

Directory Navigation

- `pwd` – Show current directory.
- `ls -l` – List files with details.
- `cd` – Change directory.

Examples:

```
pwd
```

```
ls -l
```

```
cd data && pwd && cd ..
```

File Creation & Viewing

Creating and Viewing Files using **touch** and **cat**

- `touch data2.csv` `#touch` → Create file
- `ls -l data2.csv` `#ls -l` → File information
- `wc -l data2.csv` `#wc -l` → Count lines
- `head -3 data2.csv` `#head` → First few lines
- `cat data2.csv` `#cat` → Display file

Searching with grep

grep – Search Text in File

Meaning

- **g**lobal **r**egular **e**xpression **p**rint
- **S**earch for **t**ext or **p**atterns in **f**iles.

Example:

```
grep pass data.csv
```

```
grep fail data.csv
```

```
grep CSE data.csv
```

```
grep EE data.csv
```

Advanced grep Options

- `grep -c pass data.csv` `#-c` → Count matches
- `grep -n fail data.csv` `#-n` → Show line numbers
- `grep -i PASS data.csv` `#-i` → Ignore case
- `grep -v Fail data.csv` `#-v` → Invert match
- `grep -E "CSE|EE" data.csv` `#-E` → Regular expressions, | is OR operator
- `grep -rn Pass .` `#-rn` → Recursive search

Editing Text using sed

sed – Stream Editor

- **Stream Editor**
- Edit text line by line.
- Replace, delete, or modify text.
- By default, does **not** modify the original file.
- **g** (global) replaces all matches.

Example:

```
sed 's/pass/PASS/' data.csv
```

```
sed 's/fail/FAILED/' data.csv
```

```
sed 's/pass/PASS/g' data.csv | head -3
```

Data Processing using awk

awk – Pattern Scanning & Data Processing

- Named after its creators: **A**ho, **W**einberger, **K**ernighan
- Process structured text.
- Work with rows and columns.
- **-F**, Use comma delimiter.

Example: Extracting Columns

```
awk -F, '{print $1}' data.csv
```

```
awk -F, '{print $1,$5}' data.csv
```

```
awk -F, '$5=="fail" {print $1}' data.csv
```

awk + sort + uniq - Counting Values

Example:

```
awk -F, '{print $2}' data.csv | sort | uniq -c
```

```
awk -F, 'NR>1 {print $5}' data.csv | sort | uniq -c
```

About

- Sort data.
- Count unique values.
- **NR>1** skips header.

File Search

Example:

```
find . -name "main.py"
```

About

- Search file.
- Display absolute path.

Practice questions:

Question 1: Directory and File Management

Create the following directory structure.

cs699/

├── data/

├── scripts/

├── backup/

└── reports/

Perform the following tasks:

1. Create two files named `students.csv` and `marks.csv` inside the `data` directory.
2. Create a shell script named `analysis.sh` inside the `scripts` directory.
3. Copy `students.csv` into the `backup` directory.
4. Rename `marks.csv` to `semester_marks.csv`.
5. Display the complete directory structure and list the contents of every directory.

Question 2: Linux File Analysis

Create a CSV file named `students.csv` containing at least 4 student records with the following columns: Name, Department, Hostel No, Room no

Using Linux commands only, answer the following:

1. Display the contents of the file.
2. Count the total number of student records.
3. Sort the file such that students sharing the hostel room are contiguous
4. Display the count of students from different departments

Question 3: Remote Login and File Transfer

Using Linux command-line tools, perform the following tasks:

1. Connect to mars@cse.iitb.ac.in (remote Linux server) using SSH.
2. Display your current working directory on the server.
3. Create a directory named `cs699_lab`.
4. Copy a local file named `students.csv` to the remote directory using `scp`.
5. Verify that the file was copied successfully.
6. Logout from the server.

Write all commands used in a shell script